Universidade do Minho

Ano Letivo: 2025/26

Databases

Logic Modeling
Normalization



Summary

1 Normalization

2 Logic Modeling - SQLDeveloper

Bibliography:

- Connolly, T., Begg, C., Database Systems, A Practical Approach to Design, Implementation, and Management , Addison-Wesley, 4a Edição, 2004. **(Chapters 14 and 17)**
- Belo, O., "Bases de Dados Relacionais: Implementação com MySQL", FCA – Editora de Informática, 376p, Set 2021. ISBN: 978-972-722-921-5.

Normalization Theory

Data Normalization

Data normalization is a method used to organize information in a relational database. When applied, the attributes of a data model are structured in a way that maintains consistency between entities, reducing or eliminating data redundancy. This process helps improve storage efficiency, as well as the integrity and scalability of the database. In this database systems unit, the main concepts related to normalization will be covered, with special attention given to the normalization process, the different normal forms, functional dependencies, and finally, the possible “denormalization” of a database.

“There is, in fact, a very simple elimination procedure which we shall call normalization. Through decomposition nonsimple domains are replaced by ‘domains whose elements are atomic (nondecomposable) values.’”
(Codd, 1970)

Normalization Theory

Data Normalization

Data normalization is based on the analysis of primary keys and the functional dependencies of all its attributes.

It is a progressive process, built on a series of steps, each corresponding to a specific normal form with increasingly strict validation criteria.

Through its application, the attributes of a given data model are organized to ensure the cohesion of the involved entity types, minimizing or even eliminating data duplication, and improving storage efficiency, data integrity, and scalability.

Normalization Theory

Data Normalization

In a data normalization process, it is important to consider some fundamental principles:

- Each record in a table should represent an entity or an instance of a relationship.
- The connection or reference to other entities should be made only through foreign keys.
- Whenever possible, the attributes of entities and those of relationships should be kept separate.
- The structure of the database schema should be organized in a way that can be understood and explained table by table.
- The semantics of the attributes should be clear and easy to understand.
- Mixing attributes from different entities can cause several problems, such as increased storage space due to redundant information or anomalies in data updates.

Normalization Theory

Data Normalization

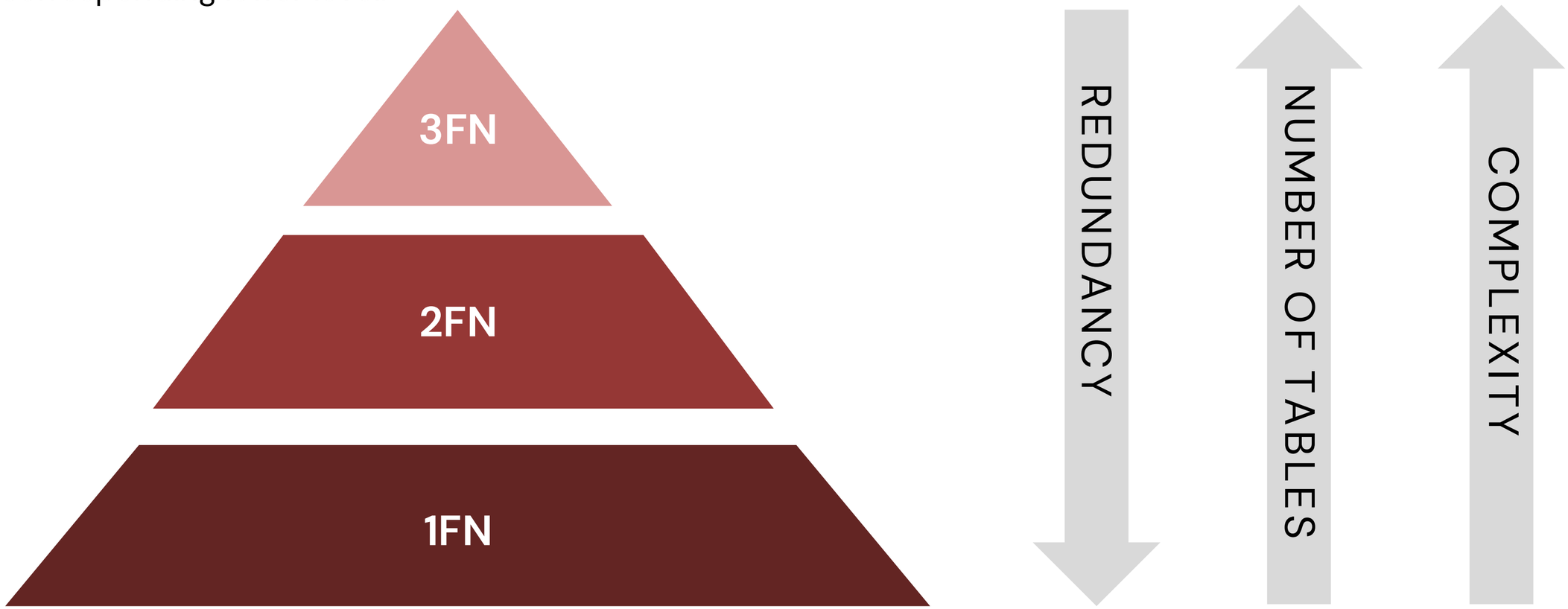
In the relational model, there are specific techniques used to evaluate the level of efficiency and organization of a database. These techniques are known as Normal Forms (NF). To achieve these forms, there are specific algorithms that allow a database to be transformed or reorganized, ensuring that it meets the requirements of each normal form.

- **1NF – First Normal Form**
- **2NF – Second Normal Form**
- **3NF – Third Normal Form**
- BCNF – Boyce–Codd Normal Form
- 4NF – Fourth Normal Form
- 5NF – Fifth Normal Form
- DKNF – Domain-Key Normal Form
- 6NF – Sixth Normal Form

Normalization Theory

Normal Forms

The normalization process is progressive; that is, each higher level of normalization is a subset of the corresponding lower level.



Normalization Theory

First Normal Form – 1FN

A relation is said to be in 1NF if:

1. It has a primary key.

2. All its attributes are atomic. Attributes that implicitly encode sub-attributes (composite attributes) or multivalued attributes are not allowed.

3. It does not contain repeating groups of data.

In practice, we can say that a relation is in 1NF if the intersections between columns (attributes) and rows (records) contain a single value — an atomic value.

Normalization Theory

First Normal Form – 1FN

Application of 1FN:

Step 1: One of the candidate keys is chosen for the primary key.

Step 2: Multivalued attributes are converted into new relations with a foreign key referencing the primary key of the original table.

Step 3: Each composite attribute is mapped into several atomic sub-attributes.

Normalization Theory

Second Normal Form – 2FN

A relation is said to be in Second Normal Form (2NF) if:

- The relation is also in 1NF.
- All its non-prime attributes are fully dependent on the primary key. That is, there must be no partial dependencies. An attribute is considered non-prime when it is not part of a primary key.

Normalization Theory

Second Normal Form – 2FN

What is a functional dependency?

Functional dependencies determine how data can be interpreted and related, and they allow the specification of formal measures regarding the correctness of relational schemas.

In practice, a functional dependency A1 → A2 between two sets of attributes of a relation means that:
For each value of A1, there is only one possible value for A2. Therefore, A2 is said to be functionally dependent on A1, or that A1 functionally determines A2.

Equal values of A1 determine equal values of A2.

Normalization Theory

Second Normal Form – 2FN

Applying functional dependency analysis:

Students(<u>id_aluno</u>, nome_aluno, cod_curso, nome_curso)

Grades(<u>id_aluno, cod_dis</u>, nome_dis, nota, cod_prof, nome_prof, dep_prof)

Dependency Diagram

id_aluno → nome_aluno, cod_curso

cod_curso → nome_curso

{id_aluno, cod_dis} → nota

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

The Students table is in the 2FN, but the Grades is not!

Normalization Theory

Second Normal Form – 2FN

Dependency Diagram

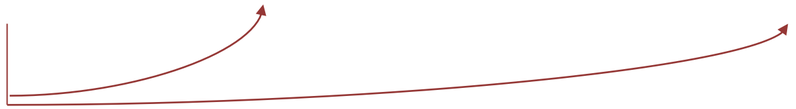
{id_aluno, cod_dis} → nota

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

Grades(<u>id_aluno, cod_dis</u>, nome_dis, nota, cod_prof, nome_prof, dep_prof)

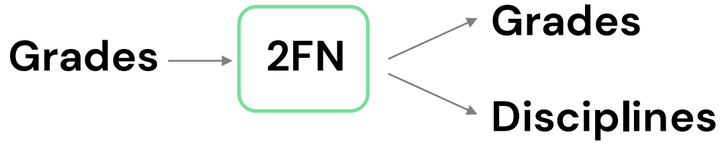
id_aluno	cod_dis	nome_dis	nota	cod_prof	nome_prof	dep_prof
001	D01	Bases de Dados	16	P01	Maria do Carmo	Dep. Informática
001	D02	Criptografia	12	P02	Paulo Gomes	Dep. Informática
002	D01	Bases de Dados	17	P01	Maria do Carmo	Dep. Informática
002	D03	Lógica Computacional	14	P03	Tiago Pinho	Dep. Sistemas
...	...	...	...	...	...	...



Partial Dependence

Normalization Theory

Second Normal Form – 2FN



Subjects(<u>cod_dis</u>, nome_dis, cod_prof, nome_prof, dep_prof)

Grades
<u>id_aluno</u>
<u>cod_dis</u>
nota

(001, 'D01', 16)
(001,'D02', 12)
(002,'D01', 17)
(002,'D03', 14)
...

Subjects
<u>cod_dis</u>
nome_dis
cod_prof
nome_prof
dep_prof

Normalization Theory

Third Normal Form – 3FN

A relation is said to be in 3NF if:

- The relation is also in 1NF and 2NF.
- All its non-primary-key attributes are mutually independent, meaning there are no transitive functional dependencies. In other words, in a 3NF relation, all attributes depend solely on the primary key.

In practice, this means that attributes that do not depend on the primary key should be “removed” from the relation — that is, they should be transferred to another table.

Normalization Theory

Third Normal Form – 3FN

In practice, the functional dependencies A1 → A2, A3 and A3 → A4 among the attributes of a relation mean that:

- There is a transitive functional dependency between A1 and A4. In other words, the attributes that are not part of the primary key are not mutually independent.

Applying functional dependency analysis to the previous case study:

Students(id_aluno, nome_aluno, cod_curso, nome_curso)

Grades(id_aluno, cod_dis, nota)

Subjects(cod_dis, nome_dis, cod_prof, nome_prof, dep_prof)

Dependency Diagram

id_aluno → nome_aluno, cod_curso

cod_curso → nome_curso

{id_aluno, cod_dis} → nota

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

The Students and Subjects table are not in 3FN!

Normalization Theory

Third Normal Form – 3FN

Dependency Diagram

id_aluno → nome_aluno, cod_curso

cod_curso → nome_curso

Students(id_aluno, nome_aluno, cod_curso, nome_curso)

id_aluno	nome_aluno	cod_curso	nome_curso
001	João Ferreira	C01	MIEI
001	João Ferreira	C01	MIEI
002	Rita Abreu	C01	MIEI
002	Rita Abreu	C01	MIEI
...	...	...	...

Transitive Dependence

Normalization Theory

Third Normal Form – 3FN

Dependency Diagram

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

Subject(cod_dis, nome_dis, cod_prof, nome_prof, dep_prof)

cod_dis	nome_dis	cod_prof	nome_prof	dep_prof
DO1	Bases de Dados	PO1	Maria do Carmo	Dep. Informática
DO2	Criptografia	PO2	Paulo Gomes	Dep. Informática
DO1	Bases de Dados	PO1	Maria do Carmo	Dep. Informática
DO3	Lógica Computacional	PO3	Tiago Pinho	Dep. Sistemas
...	...	...	...	...



Transitive Dependence

Normalization Theory

Third Normal Form – 3FN





Students(id_aluno, nome_aluno, cod_curso)

Students
id_aluno
nome_aluno
cod_curso

Subjects (cod_dis, nome_dis, cod_prof)

Subjects
cod_dis
nome_dis
cod_prof

Courses(cod_curso, nome_curso)

Courses
cod_curso
nome_curso

Teachers(cod_prof, nome_prof, dep_prof)

Teachers
cod_prof
nome_prof
dep_prof

Teoria da Normalização

➡ Terceira Forma Normal – 3FN

Students(<u>id_aluno</u>, nome_aluno, cod_curso)

Students
<u>id_aluno</u>
nome_aluno
cod_curso

Subjects
<u>cod_dis</u>
nome_dis
cod_prof

Courses(cod_curso, nome_curso)

Courses
<u>cod_curso</u>
nome_curso

Dependency Diagram

id_aluno → nome_aluno, cod_curso

cod_curso → nome_curso

{id_aluno, cod_dis} → nota

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

Teachers
<u>cod_prof</u>
nome_prof
dep_prof

As relações encontram-se na 3FN!

PHASE 4: Logical Modeling

MySQL Workbench

1) After installation, the GUI will open with the configuration to the MySQL server already done (marked in the figure)

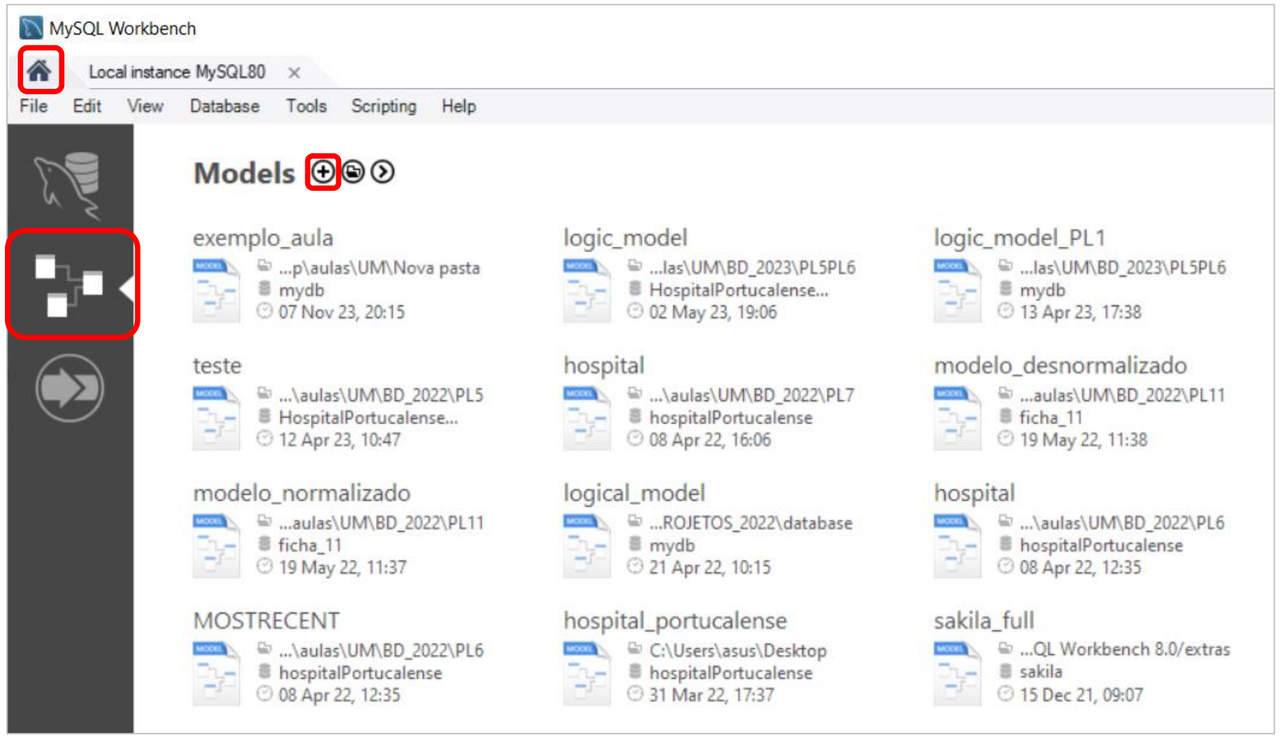


NOTE: If the connection does not appear, it is likely that you missed or missed any steps in the installation guide

PHASE 4: Logical Modeling

MySQL Workbench

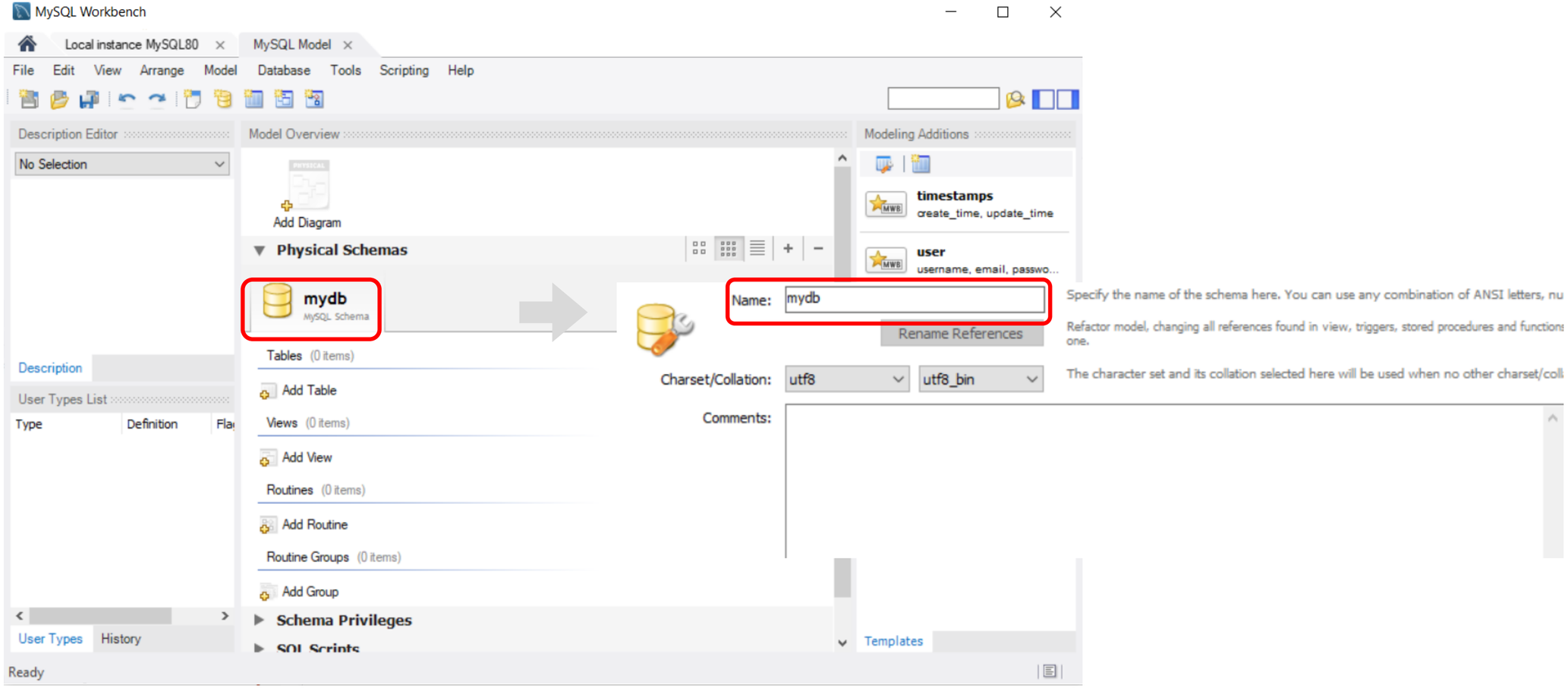
Go to the home menu and click on the "Models" tab to create a new layout.



PHASE 4: Logical Modeling

MySQL Workbench

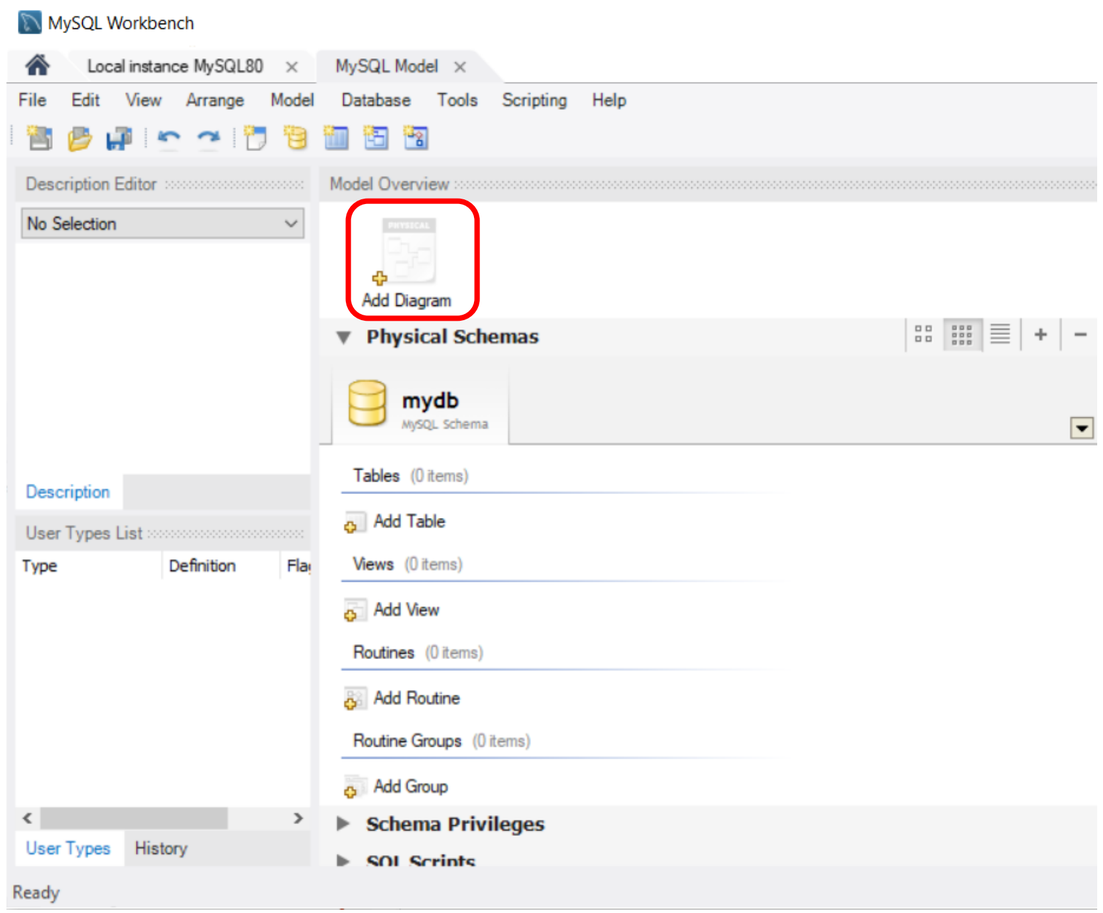
Workbench then creates a new schema named 'mydb'. To change the name of the scheme, just double-click on 'mydb'

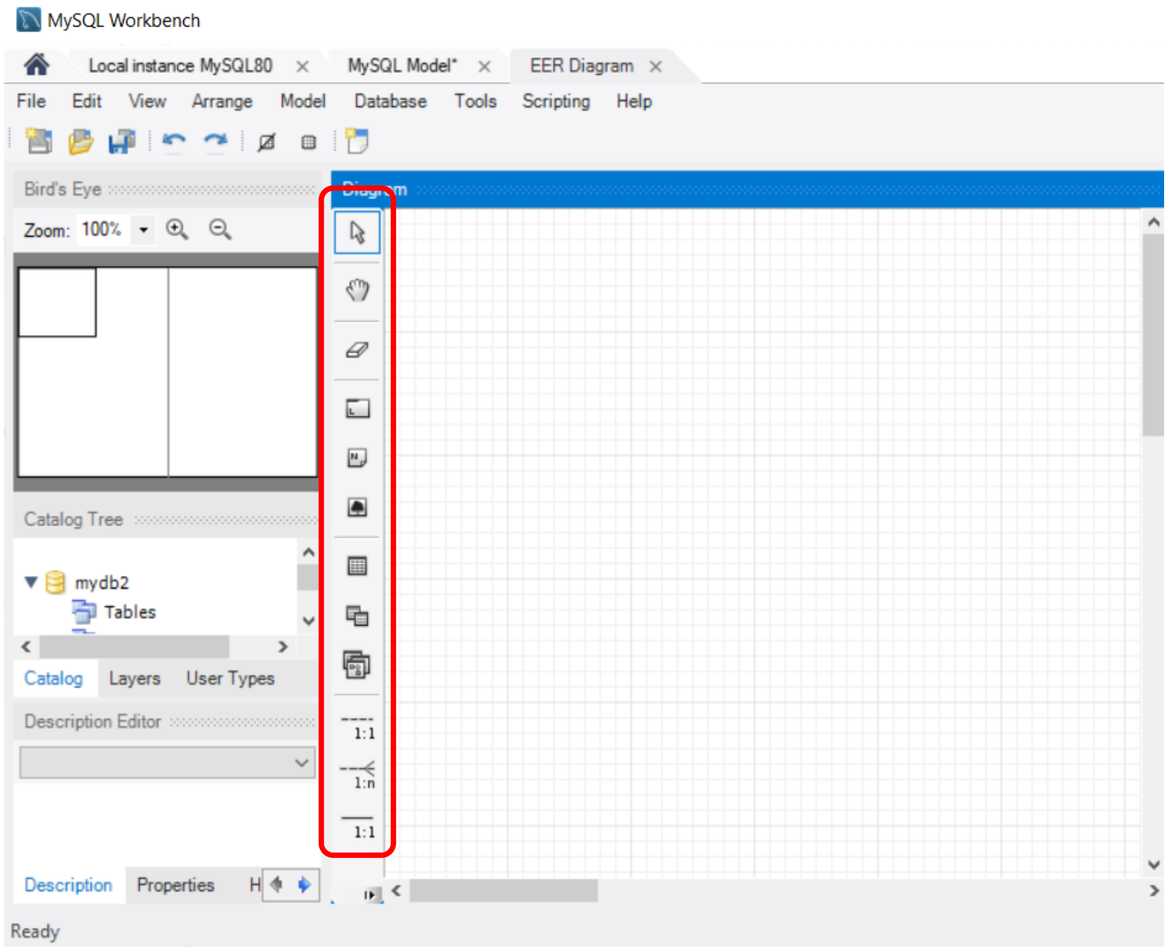


PHASE 4: Logical Modeling

➡ MySQL Workbench

Once you have set up the name, click on the 'Add Diagram' button. A new window is created called 'EER Diagram'.





Logic Modeling - MySQL

When we are building the logical data model in MySQL, it is important to take into account the following aspects:

- Relationship Type:



Identifying relationships (full line) When the primary key of the parent entity is included in the primary key of the child entity.

 - Foreign key and primary key.



Non-identifying relationships(dashed line) When the parent entity's primary key is included in the child entity, but not as part of its primary key.

 - Foreign key NOT NULL – mandatory participation in the conceptual model

 - Foreign key – optional participation in the conceptual model

- Relationship direction:

Relationships must start at the table that must allocate the foreign key.

Logic Modeling - MySQL

- Default/default values: Should be used if you want to consider a default value.



- PK (Primary Key), NN (Not Null), UQ (Unique Index), B (Binary), UN (Unsigned), ZF (Zero Fill), AI (auto increment), G (generated)

  - PK – should be used for attributes that are primary key;
  - NN – must be used on all primary key attributes and all attributes that cannot be NULL;
  - UQ - should be applied whenever there are candidate keys, so that there are no duplicate values in the table;
  - UN - defines that negative values cannot be entered in this column.
  - ZF - Fills the value defined for the field with zeros up to the display width specified in the column definition.
  - AI – should be used to automatically generate when a new record is inserted into a table.
  - G – Must be used to generate attributes from others using an expression.

Data Types in MySQL

Alphanumeric Data

https://dev.mysql.com/doc/refman/8.0/en/data-types.html

VARCHAR (variable-length strings) vs. CHAR (fixed-length strings)

Valor	CHAR(4)		VARCHAR(4)	
''	'____'	4 bytes	''	1 byte
'AB'	'AB__'	4 bytes	'AB'	3 bytes
'ABC'	'ABC_'	4 bytes	'ABC'	4 bytes
'ABCD'	'ABCD'	4 bytes	'ABCD'	5 bytes

VARCHAR uses an additional 1 or 2 bytes of memory for size or to mark the end of data.

To store longer text:
- TEXT
- TINYTEXT
- MEDIUMTEXT
- LONGTEXT

Data Types in MySQL

Alphanumeric Data

https://dev.mysql.com/doc/refman/8.0/en/data-types.html

- The ENUM type is a string object whose value is selected from a set of allowed values that are explicitly defined at column creation time.

EXAMPLE:

```
priority ENUM('Not Urgent', 'Not Very Urgent', 'Urgent', 'Very Urgent', 'Emerging') NOT NULL );
```

The priority column will only accept the insertion of one of the five defined values. MySQL maps each enumeration member to a numeric index. In this case, 'Not Urgent', 'Not Very Urgent', 'Urgent', 'Very Urgent' and 'Emergent' are mapped to 1, 2, 3, 4 and 5 respectively.

Data Types in MySQL

Date/Time Data

https://dev.mysql.com/doc/refman/8.0/en/data-types.html

Data Type	Notation
DATE	YYYY-MM-DD
TIME	hh:mm:ss
DATETIME*	YYYY-MM-DD hh:mm:ss
TIMESTAMP**	YYYY-MM-DD hh:mm:ss
YEAR	YYYY

* The supported range ranges from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'.
** The supported range ranges from '1970-01-01 00:00:01' to '2038-01-19 03:14:07'.

Data Types in MySQL

Date/Time Data

https://dev.mysql.com/doc/refman/8.0/en/data-types.html

- Fixed-Point Types (Exact Value) - DECIMAL

DECIMAL types store exact numerical data values. This type of data is used when it is important to preserve exact accuracy, for example with monetary data.

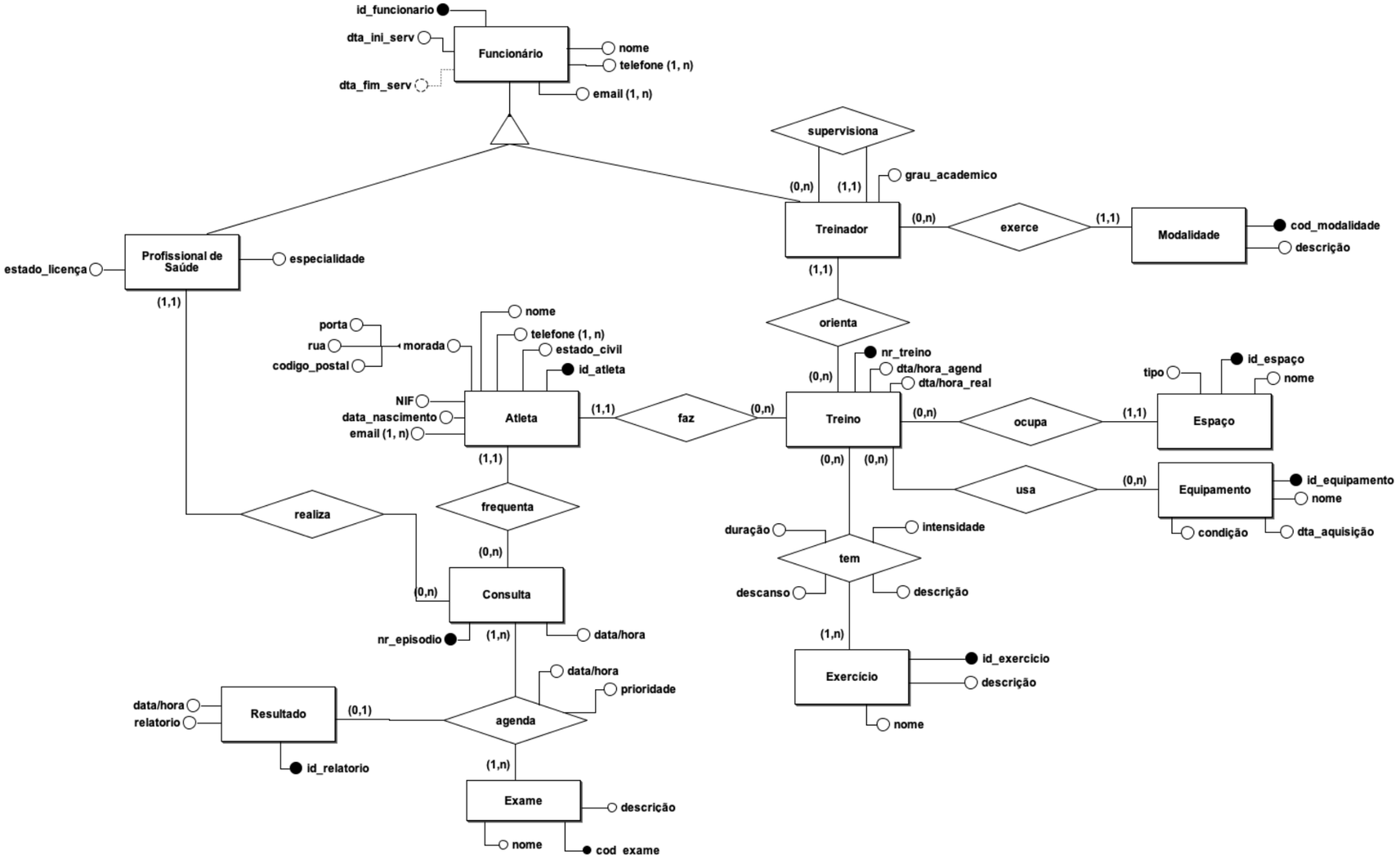
DECIMAL(n,m)

n – Precision - represents the number of significant digits that are stored.
m – Scale - represents the number of digits that can be stored after the decimal point.

Example: 105,98€ -> DECIMAL (5,2)

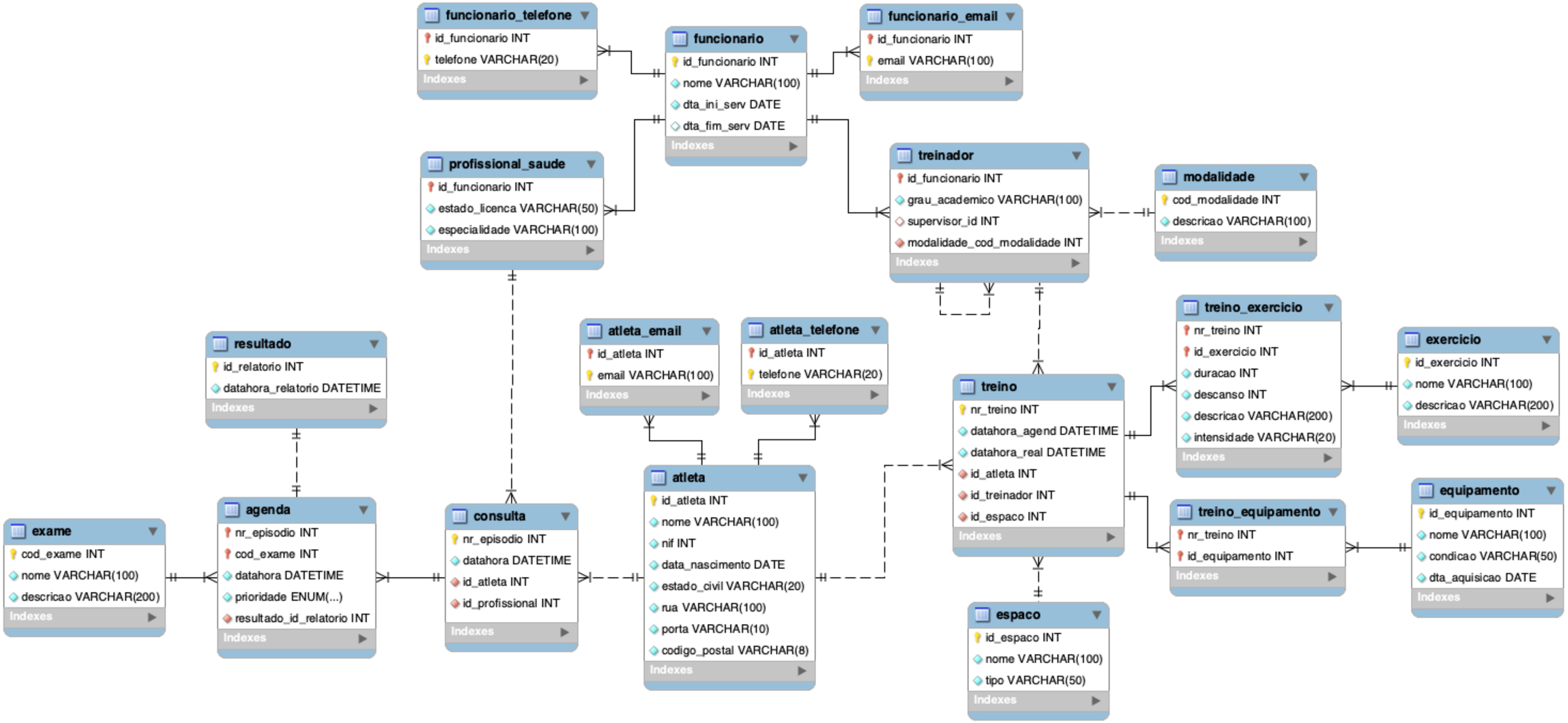
Logical Modeling

Case Study



Logical Modeling

➡ Case Study



Universidade do Minho

Ano Letivo: 2025/26

Bases de Dados

Modelação Lógica (continuação)
Normalização



António Abelha | abelha@di.uminho.pt